

Helix OTA — Cuttlefish real-A/B bring-up RUNBOOK on nezha (operator-driven privileged steps)

Field	Value
Revision	1
Last modified	2026-06-22T20:30:00Z
Status	runbook-ready — EXECUTE this to perform the real Cuttlefish A/B + auto-rollback run on nezha; integration-pending until run with captured slot-flip/rollback evidence The exact, ordered, operator-vs-agent step split for the real Cuttlefish (cvd) Tier-2 OTA run on nezha. nezha has no passwordless sudo, so every privileged step is fenced as an OPERATOR PRIVILEGED STEP the operator runs by hand; the agent drives every rootless/unprivileged step. This is a recipe to EXECUTE — every device-mount, the Virtual-A/B-vs-legacy variant, and the exact corrupt-slot mechanism are still UNCONFIRMED: per §11.4.6 and are established-as-FACT at run time, never guessed here. HONEST BOUNDARY: this runbook does NOT claim a real-A/B PASS. A real-A/B PASS is earned only by running the steps below and capturing slot-flip + auto-rollback evidence (§11.4.107/§11.4.108/§11.4.69).
Status summary	
Authority	Helix OTA control-plane / device-integration team docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md (the §11.4.161 documented exception this runbook executes under); docs/design/CUTTLEFISH_TIER2.md (the Tier-2 recipe + sources); tests/emulator/tier2_cuttlefish_ab.sh (the A/B + auto-rollback validation the
Related	

Field	Value
	agent drives); containers/pkg/cuttlefish/ (cvd-lifecycle wrapper + Containerfile + entrypoint.sh)

0. What this runbook is (and is NOT)

This is the **decision-A deliverable**: the EXACT step-by-step for the real Cuttlefish A/B run on nezha. It is the operational complement to docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md (which *authorises* the container privilege) and docs/design/CUTTLEFISH_TIER2.md (which holds the cited research).

- **What it is**: an ordered checklist where the **operator** runs the few privileged steps (nezha has no passwordless sudo) and the **agent** drives everything rootless/unprivileged around them.
- **What it is NOT**: a claim that Tier-2 ran. No PASS is asserted here. Real-A/B fidelity is earned only by executing §§4–6 below and capturing the slot-flip + auto-rollback evidence. Until then this path is runbook-ready + operator-privileged-launch-pending — not a green.

Every step is marked **[OPERATOR]** (privileged, run by hand) or **[AGENT]** (rootless, the agent drives). The **only** privileged surface is module-load + device-node creation + the `--privileged` container run (§5 scope-narrowing in CUTTLEFISH_ROOTFUL_EXCEPTION.md).

1. Prereqs (assets staged + image built)

1.1 Verified nezha device state (FACT — host probe 2026-06-22)

Node / module	State on nezha	Consequence
/dev/kvm	present, world-rw	KVM available — passthrough OK
/dev/vhost-vsock	present, world-rw	vsock VMM path OK
/dev/vhost-net	present, world-rw	guest-net VMM path OK
/dev/net/tun	present, world-rw	tap-device backing OK
/dev/vsock	ABSENT	the <code>--device /dev/vsock</code> line is conditional (§2.1)
vhost_vsock / vhost_net kernel modules	not in lsmod — consistent with built-in to the Linux 6.12 kernel	UNCONFIRMED: built-in vs unloaded — VERIFY at §2.1 before the first launch (do not guess)

nezha is a Linux x86_64 host with KVM (62 GB RAM). Host arch x86_64 → Cuttlefish target aosp_cf_x86_64_only_phone-userdebug (no credentials, public ci.android.com) [see CUTTLEFISH_TIER2.md Sources].

1.2 Staged assets (FACT — ~/cf-staging/ on nezha)

Asset	Size	Role
~/cf-staging/cvd-host-package.tar.gz	~898 MB	the cvd host package — extracts to ./bin/launch_cvd, ./bin/adb, ./bin/stop_cvd, ./bin/update_device.py
~/cf-staging/img.zip	~1.16 GB	the device-image zip (super.img / system.img etc.) for the matching build

Both come from the **SAME** aosp_cf_x86_64_only_phone-userdebug build (the host package + device image **MUST** be from one build or launch_cvd mismatches). **UNCONFIRMED**: the exact build id of the staged pair — confirm at §3 by reading the extracted bin/cvd_host_bugreport / manifest, never assume.

1.3 Container image build — [AGENT], rootless

Building the Cuttlefish container image needs **no privilege** (ordinary rootless podman build) per CUTTLEFISH_ROOTFUL_EXCEPTION.md §5(1) + the Containerfile header. The agent runs this on nezha when it is free:

```
# [AGENT] rootless — on nezha, from the containers submodule root
cd ~/helix_ota/containers # or wherever the checkout lives on
                             nezha
podman build \
  --build-arg BASE_IMAGE=debian:12 \
  -f pkg/cuttlefish/Containerfile \
  -t cuttlefish:latest .
```

UNCONFIRMED: whether the staged ~/cf-staging assets are baked into the image (via BUILD_ID/CF_IMG_URL build args) or mounted at run time (§4). This runbook uses the **mount-at-run-time** path (§4) — simpler, lets the agent re-stage without rebuilding. The image therefore only carries the cuttlefish host packages; the guest images come from the bind-mounted ~/cf-staging.

Note (mount vs endpoint-fetch): the endpoint.sh can fetch assets from CF_IMG_URL/CF_HOST_PKG_URL at run time, but since the assets are **already staged locally** at ~/cf-staging, this runbook bind-mounts them (-v ~/cf-staging:/staging) and extracts in-container — no re-download.

2. OPERATOR PRIVILEGED STEPS — run by hand on nezha

OPERATOR PRIVILEGED STEPS — nezha has NO passwordless sudo. The operator runs the commands in this section by hand (each needs sudo). The agent CANNOT run these and will WAIT (honest §11.4.21 block) until they are done. These are the **only** privileged steps; everything else is rootless. Authorised by docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md (§11.4.161 documented exception); high-blast-radius → operator-gated for first use (§11.4.101/§11.4.21).

2.1 [OPERATOR] Verify / load vhost modules + create /dev/vsock if absent

First **verify** whether the vhost modules are built-in (so modprobe is a no-op) vs loadable — do not guess (§11.4.6):

```
# [OPERATOR] verify-before-load (FACT-gathering, no change if built-in)
ls -l /sys/module/vhost_vsock /sys/module/vhost_net 2>/dev/null
# present => built-in or loaded
modinfo vhost_vsock 2>/dev/null | head
-1 # 'name:' line => loadable
module
lsmod | grep -E 'vhost_vsock|vhost_net' || echo "not listed (built-in OR unloaded)"
```

- If /sys/module/vhost_vsock AND /sys/module/vhost_net **exist** → built-in or already loaded → **nothing to load**, skip the modprobe.
- If they do **not** exist and modinfo shows a module → load them:

```
# [OPERATOR] load only if NOT already present (verified above)
sudo modprobe vhost_vsock
sudo modprobe vhost_net
```

Then handle the **absent /dev/vsock** client node. UNCONFIRMED: whether Cuttlefish on this kernel needs /dev/vsock in addition to /dev/vhost-vsock — the conservative move is to create it so the --device /dev/vsock line works; if the node still does not appear after modprobe vhost_vsock (the vsock core usually auto-creates it), create it manually:

```
# [OPERATOR] create /dev/vsock ONLY if still absent after vhost_vsock load.
# The vsock misc device is char major 10, minor 53 on mainline Linux -
# CONFIRM with `cat /sys/class/misc/vsock/dev` (prints "MAJOR:MINOR") BEFORE
# the mknod, never hardcode-guess the minor (§11.4.6/§11.4.111 resolve-by-name).
```

```
ls -l /dev/vsock 2>/dev/null && echo "already present – no mknod
needed" || {
DEV="$(cat /sys/class/misc/vsock/dev 2>/dev/null)" # e.g.
10:53
echo "vsock misc dev = ${DEV:-<unknown – investigate before
mknod>}"
# Only if DEV resolved: sudo mknod /dev/vsock c ${DEV%*:} $
{DEV##*:} && sudo chmod 666 /dev/vsock
}
```

Honest boundary (§11.4.6): prefer letting the vsock module auto-create /dev/vsock (load vsock if /dev/vsock is genuinely needed and absent: `sudo modprobe vsock`). Manual `mknod` is the fallback only when the kernel does not auto-create it; the major:minor MUST be read from /sys/class/misc/vsock/dev, never assumed.

2.2 [OPERATOR] (one-time) host group membership

If not already done (the Cuttlefish host packages add these via udev rules on a Debian host, but nezha runs the container path, so grant the launching user the groups the privileged container's `--group-add mirror`):

```
# [OPERATOR] one-time, then re-login (or reboot). Reversible via
gpasswd -d.
sudo usermod -aG kvm,cvdnetwork,render "$USER"
```

UNCONFIRMED: whether `cvdnetwork/render` groups even exist on nezha (they are created by the cuttlefish-base deb on a native install — on the container path they may be absent). **VERIFY** with `getent group cvdnetwork render kvm`; if a group is missing, the `--group-add <name>` in §2.3 simply has no host group to map — drop the missing ones from the run line rather than fail (§11.4.6).

2.3 [OPERATOR] The privileged container run

This is **the** §11.4.161 documented exception. Run it by hand (it needs `sudo` on nezha). The agent drives the *inside* of the container (§4) once it is up. Build the device-list from what §2.1 verified present:

```
# [OPERATOR] THE privileged launch on nezha (sudo – nezha has no
passwordless sudo).
# - Drop --device /dev/vsock if /dev/vsock is ABSENT (see
§1.1 / §2.1).
# - Drop --group-add <name> for any group getent showed
missing (§2.2).
# - -v ~/cf-staging:/staging bind-mounts the staged assets
(read at §4).
# - --name cf-helix stable container name the
agent execs into.
sudo podman run -d \
--name cf-helix \
--privileged \
--network host \
--device /dev/kvm \
--device /dev/vhost-vsock \
```

```
--device /dev/vhost-net \  
--device /dev/vsock \  
--device /dev/net/tun \  
--group-add kvm \  
--group-add cvdnetwork \  
--group-add render \  
-v ~/cf-staging:/staging:ro \  
cuttlefish:latest \  
sleep infinity
```

Why `sleep infinity` (not the `entrypoint launch`)? Running the container with `sleep infinity` as PID 1 keeps it **up and idle** so the **agent** can drive the `asset-extract` + `launch_cvd` + the full A/B/rollback validation **inside** it via `podman exec` (§4) — keeping the operator’s privileged surface to the single `podman run` and handing the multi-step, evidence-capturing flow to the agent. (The `entrypoint.sh foreground-launch` path is the alternative when no agent drives the inside; here the agent does.)

Confirm it is up (operator can eyeball, agent re-checks):

```
sudo podman ps --filter name=cf-helix      # STATUS "Up"
```

When `cf-helix` shows **Up**, hand back to the agent — the privileged surface is done.

3. Handoff marker

After §2.3, the operator signals the agent (e.g. drops a marker the agent polls, or simply tells the agent “`cf-helix` is up”). The agent’s §4 steps run `podman exec` into `cf-helix`; if `sudo podman exec` is required (rootful container owned by root), that single `exec` invocation is the one place the agent needs the operator to have granted it (or the operator runs the agent’s §4 driver script under `sudo` once). **UNCONFIRMED:** whether the agent can `podman exec` into a root-owned container without `sudo` on `nezha` — establish at handoff (try `podman exec cf-helix true`; if it fails with a permission error, the operator runs the §4 driver under `sudo`).

4. AGENT STEPS — drive the run inside the container (rootless where possible)

[AGENT] Everything in this section the agent drives, via `podman exec` into the already-privileged `cf-helix` container (or, if `exec` needs root on `nezha`, the operator runs the agent’s §4 driver script once under `sudo`). No NEW privilege is requested — the container is already privileged from §2.3.

4.1 [AGENT] Extract the staged assets inside the container

```
# [AGENT] inside cf-helix – extract host package + device image
from /staging.
podman exec cf-helix bash -lc '
    set -euo pipefail
    cd "${HOME:--/cuttlefish}"
    tar -xzf /staging/cvd-host_package.tar.gz
    ( cd "$PWD" && unzip -o /staging/img.zip )
    test -x ./bin/launch_cvd || { echo "ERROR: launch_cvd missing
        after extract" >&2; exit 1; }
    echo "extracted: $(./bin/launch_cvd --help >/dev/null 2>&1 &&
        echo launch_cvd-OK)"
'
```

UNCONFIRMED: the exact device-image filenames inside `img.zip` (`super.img` vs `system.img+vbmeta.img` etc.) and whether the build is **Virtual-A/B** or **legacy A/B** — read `getprop ro.virtual_ab.enabled` after boot (§4.3), never assume here.

4.2 [AGENT] Launch the cvd (daemon)

```
# [AGENT] inside cf-helix – daemonised launch (privileged
container already up).
podman exec cf-helix bash -lc '
    cd "${HOME:--/cuttlefish}"
    HOME="$PWD" ./bin/launch_cvd --daemon
    ./bin/adb wait-for-device
    ./bin/adb shell getprop sys.boot_completed
'
```

A non-zero exit / no `cvd-ebr` bridge here means the privileged flags from §2.3 did not take (most often a missing device or group) — that is a real FAIL surfaced in `podman logs cf-helix`, never fake-passed (§11.4.6).

4.3 [AGENT] Drive the A/B + auto-rollback validation

The agent points the existing validation harness at the running `cvd`. Two paths:

Path A (preferred) — run the project's `tier2_cuttlefish_ab.sh` harness (it detects A/B variant, applies the OTA, asserts the slot flip, then does the headline corrupt-slot **auto-rollback** — all with captured evidence + honest FAIL on any unconfirmed step):

```
# [AGENT] inside cf-helix (or against the cvd from nezha host
adb), point the
# harness at the running cvd. The harness lives in the checkout;
run it where it
# can reach ./bin/adb of the cvd. Evidence lands under docs/qa/
<run-id>/cuttlefish_ab/.
podman exec cf-helix bash -lc '
    cd "${HOME:--/cuttlefish}"
    # The cvd host package already provides bin/adb + bin/
    update_device.py;
```

```
# the harness uses ro.boot.slot_suffix + update_engine_client +
  bootctl.
HELIX_CF_DIR="$PWD/.." HELIX_CF_EVIDENCE="$PWD/evidence" \
  /path/to/helix_ota/tests/emulator/tier2_cuttlefish_ab.sh
,
```

The harness (tests/emulator/tier2_cuttlefish_ab.sh) already encodes the UNCONFIRMED: discipline (§11.4.6/§11.4.123): it **DETECTS** Virtual-A/B vs legacy A/B (ro.virtual_ab.enabled), **ATTEMPTS** the documented OTA-apply (update_device.py on an ota.zip) and the documented corrupt-slot mechanism (bootctl set-slot-as-unbootable → fallback bounded dd to the inactive system<slot> partition), and **FAILS HONESTLY** if any step does not reproduce on this host — never a fake PASS.

Path B (manual, if the harness needs host-specific adaptation) — the agent drives the same sequence by hand inside cf-helix, capturing each artifact:

1. baseline ro.boot.slot_suffix (→ getprop_before.txt)
2. build/fetch a signed OTA for aosp_cf_x86_64_only_phone-userdebug, apply via update_device.py (→ apply.log), confirm update_engine_client --follow reaches UPDATED_NEED_REBOOT (→ update_engine.txt)
3. adb reboot, assert ro.boot.slot_suffix **FLIPPED** (→ slot_after.txt) + dm-verity present (dmesg | grep -i verity → verity.txt)
4. corrupt the now-inactive slot (variant-correct: bootctl set-slot-as-unbootable or bounded dd), set it active, adb reboot, assert **AUTO-ROLLBACK** to the known-good slot (→ slot_after_rollback.txt + rollback_trace.txt)

UNCONFIRMED: (carried from CUTTLEFISH_TIER2.md, established at run time) — the exact OTA-apply invocation (update_device.py vs a cvd subcommand), the Virtual-A/B COW/snapuserd corrupt path vs the legacy direct-partition write, and bootctl shell availability. The harness attempts each and FAILs honestly; do not hardcode a guess.

4.4 [AGENT] Evidence capture

Every PASS cites a captured artifact (§11.4.69 ab_pass_with_evidence shape). The agent collects the harness's evidence dir out of the container to a durable, committed path:

```
# [AGENT] pull evidence out of the container to a durable docs/qa
  path.
RUN_ID="$(date -u +%Y%m%dT%H%M%S)-cuttlefish-ab"
mkdir -p ~/helix_ota/docs/qa/${RUN_ID}
podman cp cf-helix:/cuttlefish/evidence/. ~/helix_ota/docs/qa/${
  RUN_ID}/
# curated REPORT.md cites: slot_before, slot_after (FLIPPED),
  verity, slot_after_rollback
# (AUTO-ROLLBACK to known-good), rollback_trace – §11.4.107/
  §11.4.108/§11.4.69.
```

The durable evidence files carry the §11.4.155 project-name prefix where they are recordings; the curated REPORT.md under docs/qa/<run-id>/ is the committed proof (§11.4.83). Raw cvd logs stay gitignored (§11.4.128).

5. Honest boundary (§11.4.6) — what a green here does and does NOT mean

- A PASS from §4 means: on nezha, the real Android update_engine applied an OTA to the inactive slot, the active slot **FLIPPED** on reboot, dm-verity was present, and a deliberately-corrupted slot **AUTO-ROLLED-BACK** to the known-good slot — all with captured evidence. THAT is the real-A/B fidelity the QEMU A/B-virt tier and the NON-A/B RK3588 boards cannot reach.
 - A PASS here does **NOT** prove the RK3588 / Orange Pi 5 Max vendor HAL / U-Boot slot-switch / real-partition dm-verity (Tier-3 / OTA-004 — different boot stack); Cuttlefish is the closest hardware-free proxy, not the physical board.
 - **Until §§4–6 actually run with captured slot-flip + rollback evidence, F112 / OTA-003 stays integration-pending — NOT a real-A/B PASS.** This runbook being ready is not the run being done (§11.4.6 — “ready to run” ≠ “ran”).
 - Every **UNCONFIRMED**: above is a **verify-at-run-time** item, established as **FACT** on first bring-up (the device mounts, Virtual-A/B-vs-legacy, the corrupt-slot mechanism), never asserted from this document.
-

6. Teardown

```
# [AGENT] stop the cvd cleanly inside the container (§11.4.14
    quiescence).
podman exec cf-helix bash -lc 'cd "${HOME:-/cuttlefish}" &&
    HOME="$PWD" ./bin/stop_cvd || true'

# [OPERATOR] remove the privileged container (it was started under
    sudo on nezha).
sudo podman rm -f cf-helix
```

Reversibility (FACT, from CUTTLEFISH_ROOTFUL_EXCEPTION.md §6): no persistent host mutation beyond reversible config — the privileged window is the container’s lifetime only; the only host-state changes are the (transient) loaded kernel modules and the (reversible `gpasswd -d`) group membership. The A/B slot-switch + corrupt-slot rollback happen **inside** the virtual device, never on nezha itself (§11.4.133 — the “target” is the cvd guest, not the host).

Operator-vs-agent step split (at a glance)

Step	Who	Privileged?
§1.3 Build the Cuttlefish container image (podman build)	AGENT	no (rootless)
§2.1 Verify/load vhost_vsock/vhost_net;	OPERATOR	yes (sudo)

Step	Who	Privileged?
create /dev/vsock if absent		
§2.2 One-time group membership (usermod -aG kvm, cvdnetwork, render)	OPERATOR	yes (sudo)
§2.3 The --privileged --network host container run	OPERATOR	yes (sudo)
§3 Handoff (confirm cf-helix Up)	OPERATOR → AGENT	—
§4.1 Extract staged assets inside the container	AGENT	no (inside already-privileged container)
§4.2 launch_cvd --daemon	AGENT	no (inside already-privileged container)
§4.3 A/B apply + slot-flip + auto-rollback validation	AGENT	no
§4.4 Evidence capture to docs/qa/<run-id>/	AGENT	no
§6 stop_cvd	AGENT	no
§6 podman rm -f cf-helix	OPERATOR	yes (sudo)

The privileged surface is **exactly three operator commands** (§2.1 conditional, §2.2 one-time, §2.3 the run) + the one teardown remove — everything else is agent-driven.